EXPERIMENT 7: BUILD AN ARTIFICIAL NEURAL NETWORK (ANN) USING KERAS/TENSORFLOW FOR FASHION-MNIST CLASSIFICATION

1. DATASET SOURCE

The dataset used in this experiment is the **Fashion-MNIST dataset**, which is publicly available and widely used as a benchmark for image classification tasks.

- **Source Link:** https://github.com/zalandoresearch/fashion-mnist

2. DATASET DESCRIPTION

The **Fashion-MNIST dataset** is a collection of grayscale images representing fashion products. It is designed as a more challenging replacement for the traditional MNIST handwritten digits dataset.

Key Characteristics:

- **Total Samples:** 70,000 images
  - Training set: 60,000
  - Test set: 10,000
- **Image Size:** 28 × 28 pixels
- **Color Format:** Grayscale (pixel values range from 0 to 255)
- **Number of Classes:** 10

Feature Representation:

- Each image is represented as a **784-dimensional vector** (flattened 28×28 image).
- Pixel intensity values are numerical features.

3. MATHEMATICAL FORMULATION OF THE ALGORITHM

An **Artificial Neural Network (ANN)** consists of layers of neurons that transform input data through weighted connections and activation functions.

Forward Propagation

For a given layer (l):

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$
$$a^{(l)} = f\left(z^{(l)}\right)$$

Where:

- ($W^{(l)}$): Weight matrix
- (b): Bias vector
- (a): Activation output
- (f): Activation functios

Activation Functions

- **ReLU (Rectified Linear Unit):**

$$f(x) = \max(0, x)$$

- **Softmax (Output Layer):**

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Loss Function (Categorical Cross-Entropy)

$$L = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

Where:

- (y_i): True label
- $(\hat{y}_i)$: Predicted probability

Backpropagation

Weights are updated using gradient descent:

$$W := W - \eta \frac{\partial L}{\partial W}$$

4. ALGORITHM LIMITATIONS

Despite its effectiveness, ANN has several limitations:

1. Requires Large Data

- Neural networks perform best with large datasets.
- Small datasets may lead to **overfitting**.

2. Computationally Expensive

- Training deep networks requires significant computational power (GPU preferred)

3. Black Box Nature

- Difficult to interpret internal decision-making.
- Lack of transparency compared to models like decision trees.

4. Sensitive to Hyperparameters

- Performance depends heavily on:
  - Learning rate
  - Number of layers
  - Neurons per layer

5. Risk of Overfitting

- Especially when the model is too complex for the dataset.

6. Vanishing Gradient Problem

- In deep networks, gradients may become very small, slowing learning.

5. METHODOLOGY / WORKFLOW

The experiment follows a structured pipeline:

Step 1: Data Loading

- Load dataset using Keras API.

Step 2: Data Preprocessing

$$x = \frac{x}{255.0}$$

- Convert labels into categorical format (one-hot encoding if needed).

Step 3: Model Design

Typical ANN architecture:

- Input layer: 784 neurons
- Hidden layers:
  - Dense (128 neurons, ReLU)
  - Dense (64 neurons, ReLU)
- Output layer:
  - Dense (10 neurons, Softmax)

Step 4: Model Compilation

- Optimizer: Adam
- Loss function: Categorical Cross-Entropy
- Metrics: Accuracy

Step 5: Model Training

- Train using:
  - Batch size
  - Epochs
- Model learns patterns through iterative weight updates.

Step 6: Model Evaluation

- Evaluate on test dataset.
- Measure generalization performance.

6. PERFORMANCE ANALYSIS

Evaluation Metrics

1. Accuracy

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Observed Performance (Typical)

- Training Accuracy: ~90–92%
- Test Accuracy: ~88–90%

Interpretation

- High training accuracy indicates good learning.
- Slight drop in test accuracy suggests **minor overfitting**.
- ANN effectively captures **visual patterns** in clothing items.

Error Analysis

- Misclassifications occur between similar classes:
  - Shirt vs T-shirt
  - Sneaker vs Ankle boot

7. HYPERPARAMETER TUNING

Hyperparameter tuning improves model performance by optimizing key parameters.

Important Hyperparameters

Parameter	Description
Learning Rate	Step size for weight updates
Batch Size	Number of samples per update
Epochs	Number of training iterations
Hidden Layers	Model depth
Neurons	Model capacity

Tuning Techniques

1. Manual Tuning

- Experiment with different values.

2. Grid Search

- Try all combinations of parameters.

3. Random Search

- Randomly sample parameter combinations.

Final Observation

- Proper tuning can improve accuracy by **2–4%**
- Prevents overfitting and enhances robustness

CONCLUSION

This experiment demonstrates how an **Artificial Neural Network built using Keras/TensorFlow** can effectively classify fashion images. The model learns hierarchical feature representations and achieves strong accuracy on a complex dataset. With proper preprocessing, architecture design, and hyperparameter tuning, ANN proves to be a powerful tool for image classification tasks.